Physics Informed Deep Operator Networks for Neutron Transport

William Eivik Olsen*
Institute for Energy Technology
(Dated: July 31, 2026)

We study the effect of physics-informed training for the performance of a DeepONet on the Neutron Transport problem of a 1D slab. The main research question is to which extent physics-informed training can replace labelled training. The performance is compared for a benchmark DeepONet, a larger DeepONet and two physics-informed DeepONets. To investigate the effect of physics-informed training, the four models are trained on three datasets of different sizes. We find that among the models trained on the largest dataset, the large DeepONet performed the best. As the size of the training datasets were decreased, the physics-informed models performed better. We conclude that physics-informed training does, to some extent, replace labelled training.

I. INTRODUCTION

Global climate mitigation targets motivate the rapid expansion of low-carbon energy sources, and technological advances have renewed interest in nuclear energy as a reliable, compact, and dispatchable option. Small Modular Reactors (SMRs), typically up to 300 MW(e) per unit, promise factory fabrication and modular deployment [1]. Within this landscape, microreactors at up to roughly 20 MW(e) have emerged as particularly attractive for transportable, remote, and mobile applications, enabling reliable power in settings where conventional infrastructure is limited [2].

Maritime propulsion has become a prominent candidate for such deployment. Shipping is responsible for about three percent of global greenhouse gas emissions today and could rise substantially without decisive countermeasures. The NuProShip project [3], funded by the Research Council of Norway, surveyed more than eighty reactor concepts and identified three leading candidates for ship integration: a helium gas–cooled microreactor developed by Ultra Safe Nuclear Corporation, a molten-salt SMR by Kairos Power, and a lead-cooled SMR by Blykalla. While these technologies show promise, NuProShip highlights crew availability and cost—especially the need for highly trained nuclear operators—as a principal barrier to adoption. This challenge extends beyond maritime settings to the broader class of remote and mobile applications that motivate microreactors in the first place.

To mitigate this challenge, the development of digital twin technology for microreactors has been suggested [4]. A digital twin is a digital replica of an operating asset that can receive data from live sensors and provide physics modeling and simulation of the asset. In the nuclear context, a complete physics simulation should include the neutronics of the core, thermal-hydraulics and radiation transport [5]. An additional advantage of creating a digital twin is the ability to perform multi-physics simulations of the reactor in the approval phase. For this reason, digital twins for maritime nuclear propulsion has gained the attention of Det Norske Veritas (DNV), who recently initiated the research project MONS (Model-based Nuclear Safety) to investigate model based safety assessments of maritime nuclear vessels using multi-physics simulations. This paper is a contribution to the MONS project.

The heart of such a multi-physics model is the simulation of the reactor core itself – specifically, simulating the neutron transport and the fission chain reaction from which the core releases energy. The neutron transport equation (NTE) is notoriously difficult to solve. Traditional solution methods can be divided into *Deterministic methods*, in which simplifying assumptions are used to approximate the NTE, and *Monte Carlo methods*, in which the NTE is simulated using statistical methods. Both methods are able to provide solutions of satisfying accuracy. However, a digital twin should be able to simulate the reactor in real-time [6]. Deterministic and Monte Carlo methods both suffer from high computational costs, and are unable to solve the NTE in real-time [7–9].

The advance of machine learning methods has sparked interest in applying them to enable fast solutions of the NTE. Among them is Sahadath et al. [9], who investigated a DeepONet trained on neutron transport of a 1D slab. In this paper, we will investigate the performance of a *Physics Informed DeepONet*, first introduced by [10]. We will evaluate its performance on the same 1D problem and compare it to that of Sahadath et al.

Such a 1D model is too simple to represent the complete reactor to be considered in the MONS project. The model developed in this paper will serve as the first step towards a complete 3D reactor model.

* Also at Department of Physics, University of Oslo.

II. THEORY

A. The Neutron Transport Equation

The key problem of reactor physics is the Neutron Transport Equation (NTE). It is given by

$$\begin{aligned}\frac{1}{v}\frac{\partial\psi}{\partial t} + \hat{\Omega}\cdot\nabla\psi + \Sigma_t\,\psi(\mathbf{r},\hat{\Omega},E,t) = \\ \int_{4\pi}\int_0^\infty \Sigma_s(\mathbf{r},\hat{\Omega}'\to\hat{\Omega},E'\to E)\,\psi(\mathbf{r},\hat{\Omega}',E',t)\,dE'\,d\Omega' \\ + \frac{\chi(E)}{4\pi}\int_{4\pi}\int_0^\infty \nu\Sigma_f(\mathbf{r},E')\,\psi(\mathbf{r},\hat{\Omega}',E',t)\,dE'\,d\Omega' \\ + S_{\text{ext}}(\mathbf{r},\hat{\Omega},E,t),\end{aligned} \tag{1}$$

where $\psi(\mathbf{r},\hat{\Omega},E,t)$ is the angular neutron flux at position $\mathbf{r}$, travelling in direction $\hat{\Omega}$, with energy E, at time t. The quantity v is the neutron speed corresponding to energy E, so that $\frac{1}{v}\frac{\partial\psi}{\partial t}$ represents the time rate of change of the neutron population.

The term $\hat{\Omega}\cdot\nabla\psi$ is the streaming (leakage) term, accounting for neutrons moving through the spatial domain. The macroscopic total cross section Σ_t governs the loss of neutrons from the phase-space element due to all interactions (absorption plus scattering), so $\Sigma_t\,\psi$ is the total collision (removal) term.

On the right-hand side, $\Sigma_s(\mathbf{r},\hat{\Omega}'\to\hat{\Omega},E'\to E)$ is the double-differential macroscopic scattering cross section, describing neutrons that scatter from direction $\hat{\Omega}'$ and energy E' into direction $\hat{\Omega}$ and energy E. The integral over all incoming directions $\hat{\Omega}'$ (over the unit sphere, 4π steradians) and all incoming energies E' gives the in-scattering source.

The second term on the right side is the fission source term. The integral of the fission cross section Σ_f weighted by the neutron flux ψ and the neutrons released per fission ν gives the total number of neutrons which are emitted isotropically. The integral is multiplied by the fission spectrum $\chi(E)$, such that the final result is the number of fission neutrons at energy E.

Finally, $S(\mathbf{r},\hat{\Omega},E,t)$ is the external neutron source term.

The NTE describes the interactions of neutrons in the reactor and, importantly, the fissions they cause along with the new neutrons released from fission. A nuclear reactor is a controlled fission chain reaction, with the goal of producing as many new neutrons from fissions as the number lost through absorption or leakage. To model this, it is therefore necessary to solve the NTE.

The NTE is traditionally solved using either Monte Carlo methods or Deterministic methods. Both are computationally expensive, which is what motivates us to introduce machine learning methods to approximate solutions. However, in order to train a machine learning model, we must have training data. This can be generated by solving the NTE with traditional methods.

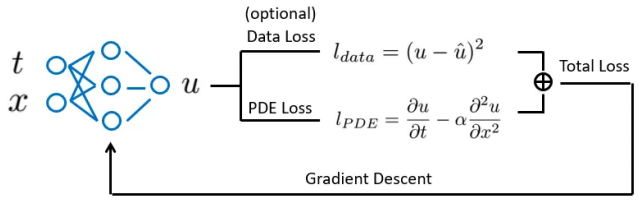

FIG. 1. Illustration of a PINN. Adapted from [11].

Given the computational expense of traditional methods, we might face a challenge to generate enough training data for our model. This motivates the use of *Physics-Informed* training, where we introduce an additional loss term that represents the governing differential equation of the problem.

B. Machine Learning Methods

1. *Physics-Informed Neural Networks*

A Physics-Informed Neural Network (PINN) is a type of deep learning model designed to solve problems governed by physical laws, specifically those described by partial differential equations (PDEs) [12]. It uses a standard neural network to approximate the unknown solution as a function of input variables. The key idea is that the network is trained not only to match available data points, but also to satisfy the governing PDE itself within the domain, as illustrated in Figure 1. This constraint is incorporated into the training process by calculating the PDE residual using automatic differentiation and minimizing it alongside the standard loss function.

An advantage of PINNs in neutronics is their ability to perform fast computations. While the training process can require significant computational resources, once trained, the model can make predictions quickly. PINNs have been studied by several authors in the context of neutronics. Prantikos et al. [13] implemented a PINN model trained on the point kinetics equations and tested its predictions using experimental data from Purdue University Reactor Number One (PUR-1). More recently, studies have explored PINNs trained with the neutron diffusion equation [14] and the neutron transport equation [15].

One disadvantage with PINNs is that they require retraining for new input boundary/initial conditions [9]. This is challenging for the maritime settings, where initial and boundary conditions are expected to vary dynamically.

2. *Deep Operator Networks*

The disadvantage we discussed with PINNs are shared with most traditional machine learning methods, as

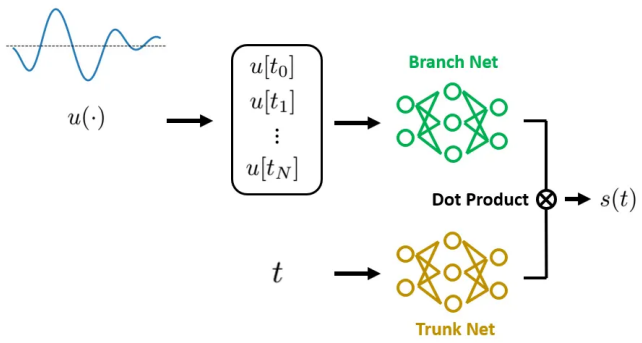

FIG. 2. Illustration of a Deep Operator Network. Adapted from [11].

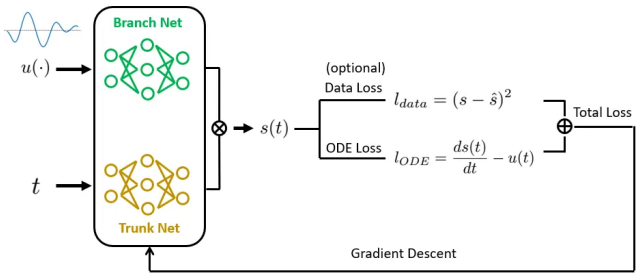

FIG. 3. Illustration of a Physics-Informed DeepONet. Adapted from [11].

they learn pointwise mapping between input-output data pairs. Deep operator networks, on the other hand, learn mappings between entire input and output function spaces. DeepONets treat inputs and outputs as functions, and aim to learn the solution operator that maps initial or boundary condition functions to corresponding solution functions [9]. This framework of learning the function mapping allows them to generalize to previously unseen conditions without the need for retraining. A DeepONet can therefore act as a sufficient surrogate model for the neutron transport equation, delivering accurate predictions for a wide range of scenarios while maintaining high generalization performance.

A DeepONet consists of two neural networks:

- A TrunkNet that processes evaluation points, such as time or spatial coordinates.
- A BranchNet that encodes discretized condition functions, such as initial and boundary conditions.

The final output is obtained through the dot product of the output of these two networks. An illustration of the DeepONet architecture is shown in figure 2.

Consider an input function $u(x)$ that can be sampled at a set of discrete positions $\{x_1, x_2, ..., x_m\}$ in the domain D_x as $[u(x_1), u(x_2, ..., u(x_m)]$. Given an operator G that acts on $u(x)$, $G(u)$ represents the resulting output function in domain D_y. Note that the domains D_x and D_y are not necessarily the same and can be entirely different. Within the domain of $G(u)$, the real value at any sampling point y_i is expressed as $G(u)(y_i)$. From the generalized universal approximation theorem for operators, for any $\epsilon > 0$, there exists positive integers n, p and continuous vector functions $\mathbf{g} : \mathbb{R}^n \to \mathbb{R}^p$ and $\mathbf{f} : \mathbb{R}^d \to \mathbb{R}^p$ such that

$$|G(u)(y_i) - \langle g(u(x_1), ..., u(x_m)), f(y_i)\rangle| < \epsilon, \qquad (2)$$

where $\langle \cdot, \cdot \rangle$ denotes the dot product defined in $\mathbb{R}^p$, and the functions g and f can be represented by any neural networks.

The approximation of $g(u(x_1), ..., u(x_m))$ is made by the BranchNet. This is done by mapping the input $[u(x_1), u(x_2, ..., u(x_m)]$ to a vector $\mathbf{b_N} = [b_1, b_2, ..., b_p]$. The TrunkNet takes an evaluation point y_i from the domain of the target operator $G(u)$ and outputs a vector $\mathbf{t_N} = [t_1(y_i), t_2(y_i), ..., t_p(y_i)]$. Finally, the output is computed by

$$G(u)(y_i) \approx \sum_{k=1}^{p} b_k(u(x_1), ...u(x_m))t_k(y_i). \qquad (3)$$

The challenge faced by DeepONets is their requirement of large annotated datasets consisting of input-output observations. Another worry for our usecase is the model's ability to extrapolate: If the reactor enters a state significantly outside the training distribution, the DeepONet predictions can become unphysical.

3. *Physics-Informed DeepONets*

We have so far introduced PINNs and DeepONets. Both have strengths that make them suitable for solving differential equations in physics, but, as we have seen, each of them have their drawbacks:

- PINNs require retraining for every new initial and boundary condition. This is problematic for the maritime case, where we expect the conditions to vary dynamically.
- DeepONets require large annotated datasets of input-output observations, which for our case means running a large amount of Monte Carlo simulations of the reactor design we wish to model.

A promising method to circumvent this issue is to combine these two methods into a single framework: A Physics-Informed DeepONet. The architecture is visualized in figure 3. In addition to the Branch and Trunk net, the model incorporates the governing differential equation in its loss function. Its loss function is given by

$$\mathcal{L} = \lambda_{\text{data}}\mathcal{L}_{\text{data}} + \lambda_{\text{PDE}}\mathcal{L}_{\text{PDE}} + \lambda_{\text{BC}}\mathcal{L}_{\text{BC}}, \qquad (4)$$

where $\mathcal{L}_{\text{data}}$ is the loss term for the labelled data, given by the mean squared error between the model predictions

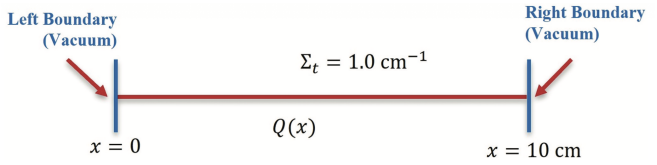

FIG. 4. Illustration of the 1D slab geometry. Adapted from [9].

and the true values. The term $\mathcal{L}_{\mathrm{PDE}}$ is the loss term for the governing PDE, and $\mathcal{L}_{\mathrm{BC}}$ is the loss term for the boundary conditions. These loss terms will be defined following the problem statement. The λ weights control the relative importance of each loss term.

The architecture was first suggested by [10], who demonstrated its effectiveness on various types of PDEs, including a diffusion model. The framework is able to combine the advantages of PINNs and DeepONets: They keep the low-data requirement of the PINN methods while being able to generalize like a DeepONet. The physics constraint makes the model more robust to handle inputs outside the training distribution. For these reasons, we believe Physics-Informed DeepONets to be the most promising architecture for our usecase.

III. PROBLEM STATEMENT

For this paper, we will consider the NTE for a simple 1D slab geometry. As we will see, this simplifies the NTE and allows us to easily generate a training dataset without using Monte Carlo or deterministic approximations. The performance of a DeepONet on this problem has been studied by [9]. We will use this model as a benchmark and develop our own Physics-Informed DeepONet model on the same problem to compare their performances.

For this specific problem, physics-informed training is not strictly necessary, since we easily can generate training data. However, the question we wish to answer in this study is *to which extent physics-informed training can replace labelled training.* This will serve as valuable insight for our future efforts of training models for more complicated 2D or 3D problems, where training data will be scarcer due to the necessity of using Monte Carlo or Deterministic methods for training data generation.

IV. METHOD

A. Neutron Transport Problem

We adapt the problem from Sahadath et al. [9]. The geometry is a 1D slab with a length of 10 cm, spatially uniform cross sections and zero incoming current boundary conditions imposed at both ends. The geometry is illustrated in Figure 4. The total cross section is assumed to be 1.0 cm^{-1}, while the absorption and scattering cross section depend on the scenario considered.

The steady-state one-group NTE for this 1D slab is given by

$$\begin{aligned} \mu\frac{\partial\psi(x,\mu)}{\partial x} + \Sigma_t\,\psi(x,\mu) &= \int_{-1}^{1} \tfrac{1}{2}\left[\Sigma_{s0} + 3\mu\mu'\Sigma_{s1}\right]\psi(x,\mu')\,d\mu' \\ &\quad + \frac{Q(x)}{2}, \\ 0 < x < X, &\quad -1 \le \mu \le 1 \\ \psi(0,\mu) = 0, &\quad 0 \le \mu \le 1 \\ \psi(X,\mu) = 0, &\quad -1 \le \mu \le 0 \end{aligned} \tag{5}$$

Here $\psi(x,\mu)$ is the angular flux at position x along the slab and direction cosine μ, where μ denotes the cosine of the angle between the neutron's direction of travel and the x-axis. The total macroscopic cross section is Σ_t, while Σ_{s0} and Σ_{s1} are the zeroth and first Legendre moments of the differential scattering cross section, respectively, with Σ_{s1} accounting for linearly anisotropic scattering. The external neutron source distribution is denoted $Q(x)$, and X is the slab length. The boundary conditions correspond to vacuum (zero incoming current) at both ends of the slab: no neutrons enter from the left at $x = 0$ for forward-traveling directions ($\mu > 0$), and none enter from the right at $x = X$ for backward-traveling directions ($\mu < 0$).

The angular flux $\psi(x,\mu)$ can be integrated over the angular variable μ to obtain the *scalar flux* $\varphi_0(x)$,

$$\varphi_0(x) = \int_{-1}^{1} \psi(x,\mu)\,\mathrm{d}x\,. \tag{6}$$

By taking the integral of the angular flux weighted by the angular variable μ, the *scalar current* $\varphi_1(x)$ is obtained,

$$\varphi_1(x) = \int_{-1}^{1} \mu\psi(x,\mu)\,\mathrm{d}x\,. \tag{7}$$

B. Data Generation

In the paper by Sahadath et al, a set of source distributions $Q(x)$ were prepared, and the 1D NTE was solved numerically for each of them to generate a training dataset for their DeepONet model. This was accomplished by fist applying spatial and angular discretization. The spatial domain was partitioned into J uniformly spaced cells using the cell-averaged finite difference method. Denoting the edges of the j'th cell by $x_{j\pm\frac{1}{2}}$, the cell width h_j and cell centre x_j are given by

$$h_j = x_{j+\frac{1}{2}} - x_{j-\frac{1}{2}}, \tag{8}$$

$$x_j = \frac{1}{2}\left(x_{j+\frac{1}{2}} + x_{j-\frac{1}{2}}\right). \tag{9}$$

Then we have the cell average p_j of a function $p(x)$ over the j'th cell defined as

$$p_j = \frac{1}{h_j} \int_{x_{j-\frac{1}{2}}}^{x_{j+\frac{1}{2}}} p(x)\mathrm{d}x. \tag{10}$$

The discrete ordinates (S_N) method is employed for angular discretization, using Gauss-Legendre quadrature with $N = 16$ angular bins. After discretizing space and angle, the 1D NTE (5) can be rewritten to

$$\begin{aligned} &\frac{\mu_n}{h_j}\left(\psi_{n,j+\frac{1}{2}} - \psi_{n,j-\frac{1}{2}}\right) + \Sigma_{t,j}\,\psi_{n,j} \\ &\quad = \frac{1}{2}\left(\Sigma_{s0,j}\,\varphi_{0,j} + 3\mu_n\Sigma_{s1,j}\,\varphi_{1,j} + Q_j\right), \\ &\quad 1 \le n \le N, \quad 1 \le j \le J \\ &\psi_{n,\frac{1}{2}} = 0, \quad 1 \le n \le \tfrac{N}{2}, \quad \mu_n > 0 \\ &\psi_{n,J+\frac{1}{2}} = 0, \quad 1 \le n \le \tfrac{N}{2}, \quad \mu_n < 0. \end{aligned} \tag{11}$$

Here n and j are the angular bin and spatial cell indices, μ_n is the cosine of the scattering angle for direction n. The angular fluxes at the right and left cell edges are denoted $\psi_{n,j+\frac{1}{2}}$ and the cell-average total cross section is $\Sigma_{t,j}$, and $\Sigma_{s0,j}$ and $\Sigma_{s1,j}$ are the cell-average zeroth and first Legendre moments of the differential scattering cross section, respectively. The cell-average external source is Q_j and the cell-average scalar flux $\varphi_{0,j}$ and current $\varphi_{1,j}$ are obtained from the angular flux via Gauss-Legendre quadrature,

$$\varphi_{0,j} = \sum_{n=1}^{N} \psi_{n,j} w_n, \tag{12}$$

$$\varphi_{1,j} = \sum_{n=1}^{N} \psi_{n,j} \mu_n w_n, \tag{13}$$

where w_n are the Gauss-Legendre quadrature weights. The cell-edge and cell-average angular fluxes are related using the diamond difference approximation,

$$\psi_{n,j} = \frac{1}{2}\left(\psi_{n,j+\frac{1}{2}} + \psi_{n,j-\frac{1}{2}}\right). \tag{14}$$

The algebraic equation (11) can be solved by making an initial assumption of $\varphi_{0,j}$, $\varphi_{1,j}$ and $\psi_{n,j}$, then updating each of them iteratively according to (11) until convergence.

The spatially varying neutron source $Q(x)$ is modeled as a sample drawn from a Gaussian random field (GRF). A GRF is a stochastic process in which the value at each spatial point is a random variable, and the joint distribution across space is multivariate normal, fully specified by a mean function $m(x)$ and a covariance kernel $C(x_i, x_j)$. The mean function sets the expected value of $Q(x)$ — physically, the expected neutron emission rate density of the internal source — at each location x, while the covariance kernel encodes how the values of $Q(x)$ at different spatial points are correlated. Following Sahadath et al. [9], the covariance is given by the squared exponential kernel

$$C(x_i, x_j) = \sigma^2 \exp\left(-\frac{(x_i - x_j)^2}{2\ell^2}\right), \tag{15}$$

where σ^2 is the variance, controlling the amplitude of the fluctuations of $Q(x)$ about its mean, and $\ell > 0$ is the length scale, which governs the spatial smoothness of the realizations — smaller values of ℓ correspond to more rapidly varying source profiles.

The kernel is evaluated at the midpoints of the $J = 100$ uniformly spaced spatial cells $x_1, x_2, \ldots, x_J$, producing a $J \times J$ covariance matrix C. A single realization of $Q(x)$ is generated by sampling from the multivariate normal distribution defined by the mean vector $M = [m(x_1), m(x_2), \ldots, m(x_J)]$ and the covariance matrix C. In practice, this is done by computing the Cholesky factorization $C = LL^\top$, drawing a standard normal vector $Z \sim \mathcal{N}(0, I)$, and forming

$$Q = M + LZ. \tag{16}$$

The result is a J-dimensional vector $Q = [Q(x_1), Q(x_2), \ldots, Q(x_J)]$ giving the source value at each cell centre. Negative entries, which are unphysical for a neutron emission rate, are clipped to zero. Repeating this procedure with independent draws of Z yields the population of source samples used to train and test the DeepONet.

A total of 500 source distribution samples $Q(x)$ were generated with a mean of 5.0 and a length scale of 0.1, divided into 100 cells.

C. Model Architecture and Training

1. *Benchmark Model*

The benchmark model was implemented according to the description of [9]. The input was the source function $Q(x)$, with the $Q(\cdot)$ array being fed as input for the Branch net and the x array fed as input for the Trunk net. The dot product of the outputs of the Branch and Trunk nets then gives the predicted scalar flux $\varphi_0(x)$. The DeepONet architecture had a branch net with layer dimensions (100, 200, 200, 100) and a trunk net with dimensions (1, 200, 200, 100). Both are fully connected feedforward neural networks with ReLU activation functions. The model was trained on

- Input data: The source distribution samples $Q(x)$ described above.
- Output data: The corresponding scalar flux $\varphi_0(x)$ for each source distribution, given by equation (6).

2. Physics-Informed DeepONet

For the Physics-Informed DeepONet, the architecture must differ from the benchmark model to incorporate the PDE loss. The benchmark model predicts the scalar flux directly, but the 1D NTE (5) is formulated in terms of the angular flux $\psi(x,\mu)$. The PDE loss term $\mathcal{L}_{\text{PDE}}$ measures how far (5) is from being satisfied by the model's predictions. It is calculated by drawing M random samples of $\tilde{\psi}_{n,j}$ and calculating the residual

$$R_j = \frac{\mu_n}{h_j}\left(\tilde{\psi}_{n,j+\frac{1}{2}} - \tilde{\psi}_{n,j-\frac{1}{2}}\right) + \Sigma_{t,j}\,\tilde{\psi}_{n,j} - \frac{1}{2}\left(\Sigma_{s0,j}\,\tilde{\varphi}_{0,j} + 3\mu_n\Sigma_{s1,j}\,\tilde{\varphi}_{1,j} + Q_j\right), \tag{17}$$

where $\tilde{\psi}_{n,j}$, $\tilde{\varphi}_{0,j}$ and $\tilde{\varphi}_{1,j}$ are the model's predictions for the angular flux, scalar flux and scalar current, respectively. The PDE loss term is then given by

$$\mathcal{L}_{\text{PDE}} = \frac{1}{M}\sum_{j=1}^{M} R_j^2. \tag{18}$$

Likewise, the boundary condition loss term $\mathcal{L}_{\text{BC}}$ measures how far the model's predictions are from satisfying the boundary conditions. It is calculated by drawing M random samples of $\tilde{\psi}_{n,j}$ at the boundaries $x = 0$, $x = 10$, and evaluating how far the model is from predicting the correct boundary condition $\psi(x = 0, \mu) = 0$ for $\mu > 0$ and $\psi(x = 10, \mu) = 0$ for $\mu < 0$. The loss term is then given by

$$\mathcal{L}_{\text{BC}} = \frac{1}{M}\sum_{j=1}^{M} \tilde{\psi}_{n,j}^2. \tag{19}$$

To implement this loss funciton, the Physics-Informed DeepONet must therefore predict the angular flux $\psi(x,\mu)$ instead of the scalar flux $\varphi_0(x)$. This forces a decision for training. The model can either:

1. Train on the same dataset as the benchmark model. The model must then predict an angular flux $\psi(x,\mu)$ and feed the angular flux into the GL quadrature (12) to calculate the scalar flux $\varphi_0(x)$.
2. Train on a separate dataset from the benchmark problem. This dataset will containt the angular flux $\psi(x,\mu)$ for each source function $Q(x)$. The two datasets will share the same input, but have different outputs.

The first option would give a true comparison to training a model on the exact same data as the benchmark problem. A potential issue with this is that this dataset will lack the dimensionality of the output predicted by the model. This might prevent the model from learning sufficiently well compared to the benchmark model, which predicts the scalar flux directly. It is therefore worth testing both training regimes for the Physics-Informed DeepONet. We therefore have one Physics-Informed DeepONet trained on

- Input data: The source distribution samples $Q(x)$ described above.
- Output data: The corresponding scalar flux $\varphi_0(x)$ for each source distribution, given by equation (6).

We will refer to this model as "PI DeepONet 1". It has an architecture with a branch net with layer dimensions (100, 250, 250, 250, 250, 100) and a trunk net with dimensions (1, 500, 500, 500, 500, 1600). Both are fully connected feedforward neural networks with ReLU activation functions. Note that the output of the trunk net is larger than the branch net. The output of the trunk net is reshaped into a (100, 16) matrix. The matrix product of the trunk and branch net then gives a vector of length 16, corresponding to the 16 angular bins for each spatial point x.

The other Physics-Informed DeepONet is trained on

- Input data: The source distribution samples $Q(x)$ described above.
- Output data: The corresponding angular flux $\psi(x,\mu)$ for each source distribution.

We will refer to this model as "PI DeepONet 2". It shares the same architecture as PI DeepONet 1. Both have loss weights $\lambda_{\text{data}} = 0.7$, $\lambda_{\text{PDE}} = 0.25$ and $\lambda_{\text{BC}} = 0.05$.

The parameters for the NTE were set to the first problem considered in [9], which corresponds to isotropic scattering. The cross sections were then given by $\Sigma_t = 1.0\ \text{cm}^{-1}$, $\Sigma_a = 0.5\ \text{cm}^{-1}$, $\Sigma_{s0} = 0.5\ \text{cm}^{-1}$ and $\Sigma_{s1} = 0\ \text{cm}^{-1}$.

To test wether physics-informed loss can replace labelled training, the models were trained on datasets of three different sizes:

1. The original dataset of 500 source distributions $Q(x)$.
2. A medium sized dataset of a subset of 100 source distributions $Q(x)$ from the original dataset.
3. A small dataset of a subset of 10 source distributions $Q(x)$ from the original dataset.

3. Large DeepONet

The architecture of the PI DeepONets were determined after a hyperparameter search to find the best performing model. Since the model that was chosen contains more parameters than the benchmark model, there is a possibility that the model performs better simply because it has more parameters. We therefore trained an additional ordinary DeepONet with corresponding architecture to the PI DeepONets; a branch net with layer dimensions (100, 250, 250, 250, 250, 100) and a trunk net with dimensions (1, 500, 500, 500, 500, 100). We will refer to this model as "Large DeepONet".

TABLE I. Test datasets for model evaluation. GRF sources are defined by length scale l, mean m and variance σ^2. The combined sources use Q_1: l=0.08, m=2.5, σ^2=0.5; Q_2: l=0.10, m=5.0, σ^2=1.0; Q_3: l=0.20, m=10, σ^2=2.0. Sinusoidal sources are $Q(x) = 12.5 + \sin(2\pi f x)$.

ID	Source type	Parameters
NS	No shift	$l = 0.10$, $m = 5.0$, $\sigma^2 = 1.0$
SS_1	Small shift	$l = 0.11$, $m = 5.5$, $\sigma^2 = 1.1$
SS_2		$l = 0.09$, $m = 4.5$, $\sigma^2 = 0.9$
SS_3		$l = 0.09$, $m = 5.5$, $\sigma^2 = 1.1$
LS_1	Large shift	$l = 0.08$, $m = 2.5$, $\sigma^2 = 0.5$
LS_2		$l = 0.20$, $m = 10$, $\sigma^2 = 2.0$
LS_3		$l = 0.30$, $m = 50$, $\sigma^2 = 2.0$
LS_4		$l = 0.50$, $m = 50$, $\sigma^2 = 2.0$
LS_5		$l = 0.80$, $m = 50$, $\sigma^2 = 2.0$
LS_6		$l = 1.00$, $m = 50$, $\sigma^2 = 2.0$
LC	Linear comb.	$a_1Q_1 + a_2Q_2 + a_3Q_3$
NLC	Nonlinear comb.	Q_1Q_2
SIN_{LF}	Sinusoidal	$f = 0.2$
SIN_{HF}		$f = 0.8$

4. *Training*

All models were trained in accordance with the training procedure of [9] in which the models were trained for 2000 epochs using the Adam optimizer. The paper did not report a learning schedule. We chose to use an exponential decay learning rate schedule, starting with a learning rate of 0.001 and a decay rate of 0.9. For the Physics-Informed DeepONets, the number of transition steps was 200 epochs. For the benchmark and Large DeepONet, the number of transition steps was 100 epochs.

D. Model Evaluation

We follow the model evaluation of [9]. The model is tested on never-seen data. That includes datasets generated with the same GRF parameters as the model was trained on, and datasets generated with small or large shifts in the GRF parameters. The test cases are summarized in Table I. For each of the NS, SS and LS cases, 200 source distributions were generated using the specified GRF parameter settings. For the rest of the cases, a single source distribution was generated for each.

The performance of the model was assessed by calculating two evaluation metrics: The coefficient of determination R^2 and the average relative error. The R^2 score measures the quality of fit of a regression model and is given by

$$R^2 = 1 - \frac{\sum_{i=1}^{N}(T_{ri} - P_{ri})^2}{\sum_{i=1}^{N}(T_{ri} - \bar{T}_r)^2}, \tag{20}$$

where T_{ri} and P_{ri} represent the true value and predicted value of the scalar flux $\tilde{\varphi}_0(x)$ and $\bar{T}_r$ is the mean of the true scalar flux $\varphi_0(x)$. The R^2 score ranges up to 1, where higher values indicate better predictive accuracy. However, a high R^2 score does not necessarily imply a causal connection or guarantee that overfitting is absent. It should therefore be reported with other error metrics. We therefore also computed the average relative error (ARE) by

$$\text{ARE} = \sum_{i=1}^{N} \frac{T_{ri} - P_{ri}}{T_{ri}}. \tag{21}$$

V. RESULTS

The results of the model evaluation are shown in Figures 5 to 10. The figures show the R^2 and ARE scores for the benchmark DeepONet, the Large DeepONet, and the two Physics-Informed DeepONets, trained on the three different datasets.

VI. DISCUSSION

The true benchmark model is the DeepONet trained on the original dataset of 500 source distributions. Its performance is shown in Figures 5 and 6. We see that it achieved an R^2 score of 0.999 ± 0.005 and ARE score of $(1.1 \pm 0.2)\%$ on the non-shifted dataset. Although this must be considered a good performance, it does not quite live up to the performance reported by Sahadath et al. [9], who achieved an R^2 score of 0.9979 ± 0.0009 and ARE of $(0.50 \pm 0.09)\%$. The discrepancy is likely due to differences in the training procedure; the learning rate schedule was not reported in the paper and may have differed from the one we used. For the rest of the tests, the pattern of the benchmark's performance is similar enough to the results reported by Sahadath et al. to be considered a valid benchmark.

For the other models trained on the original dataset, we see performances similar or better than the benchmark model. Interestingly, PI DeepONet 1 appears to perform better than PI DeepONet 2. Recall that their difference was that PI DeepONet 2 had its labelled training data in the form of angular flux $\psi(x, \mu)$, while PI DeepONet 1 had its labelled training data in the form of scalar flux $\varphi_0(x)$. In our discussion of the model architecture, we speculated that the lower dimensionality of the scalar flux might make it harder for the model to learn. However, these results suggest that the opposite is true, at least in the case of training on the original dataset. One might speculate that since the model's performance is tested in terms of only the scalar flux $\varphi_0(x)$, it is enough for the model to learn this quantity well rather than the angular flux $\psi(x, \mu)$.

Another interesting observation is that the best performing model is the Large DeepONet. This model was included in our testing to factor out the possibility that the PI DeepONets would perform better than the benchmark model simply because they had more parameters.

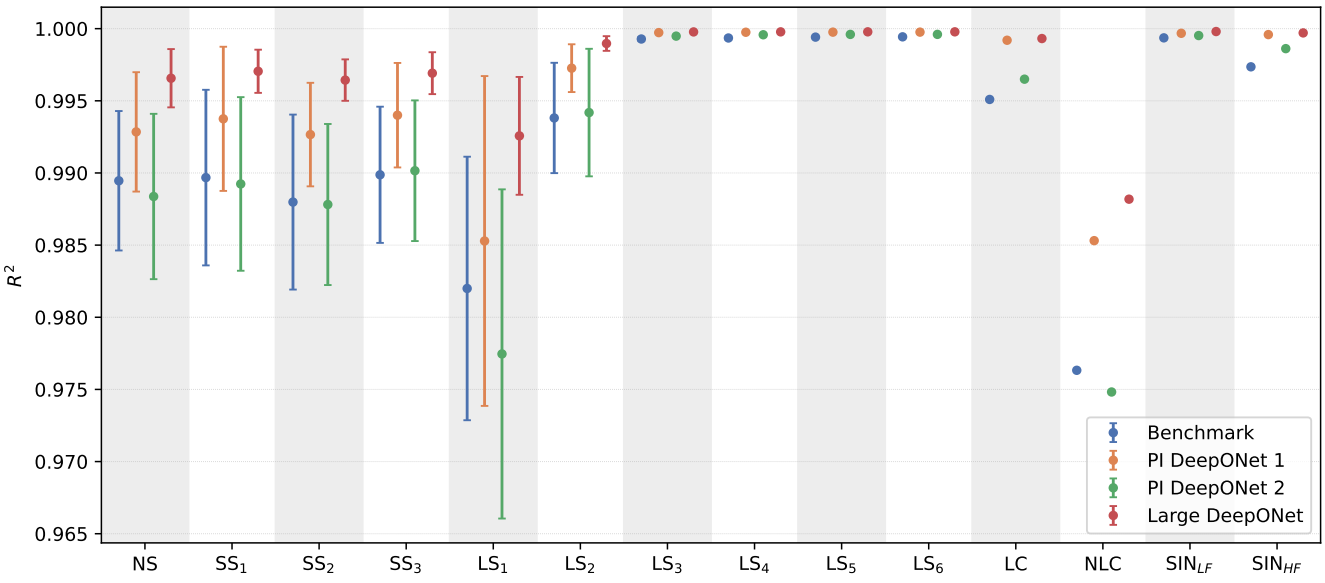

FIG. 5. R^2 score of models trained on Dataset 1, the original dataset of 500 source distributions $Q(x)$.

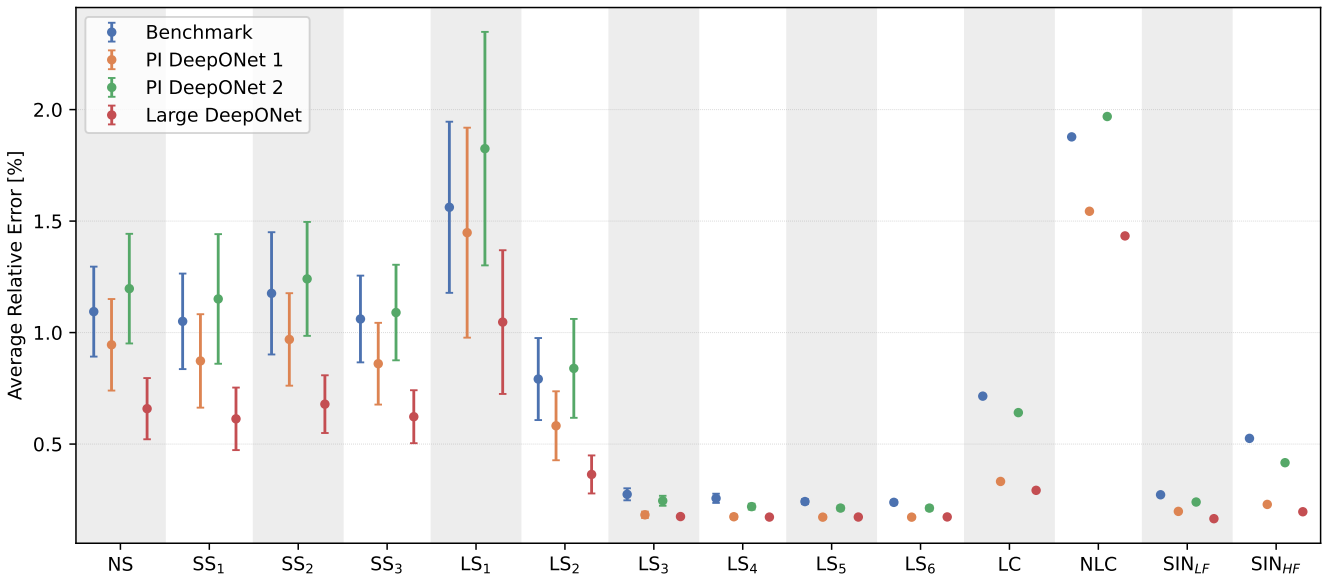

FIG. 6. ARE score of models trained on Dataset 1, the original dataset of 500 source distributions $Q(x)$.

This appears to be exactly what happened, with the PDE loss contributing negatively to the models' performance when trained on the original dataset.

For all models, we see that the small shifts in GRF parameters do not have a significant effect on the performance. Among the large shifts, the performance appears to be worst for LS1. This is likely because the source distributions in this case have the lowest mean value among the datasets, 2.5, which is half the value of the mean in the training dataset, in addition to having a lower length scale. This gives a lower, more rapidly varying scalar flux, giving higher relative errors. Likewise, for the rest of the large shifts, where the mean of the source distributions are higher, the performance appears to be better. This is likely because of the higher mean scalar flux and larger length scale, which gives smaller variations in the scalar flux. For the single test cases, the performance is worst for the nonlinear combination of two source distributions. The features discussed here are visible in the samples shown along with the prediction by PI DeepONet 1 in Figure 11.

For the models trained on Dataset 2, shown in Fig-

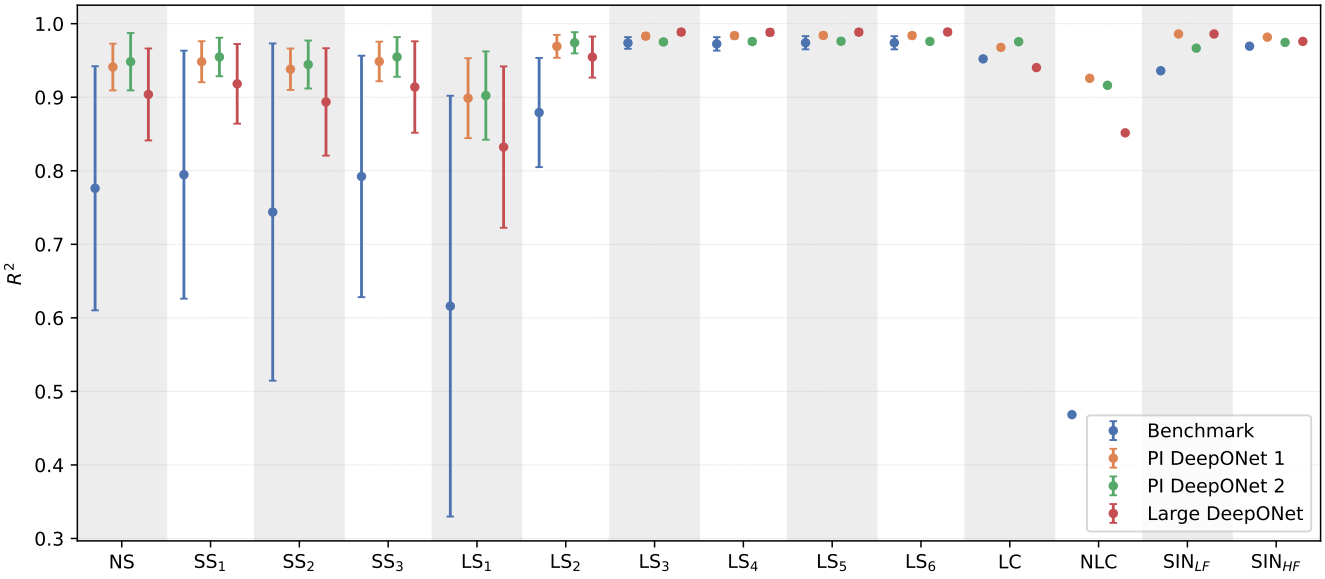

FIG. 7. R^2 score of models trained on Dataset 2, the medium sized dataset of 100 source distributions $Q(x)$.

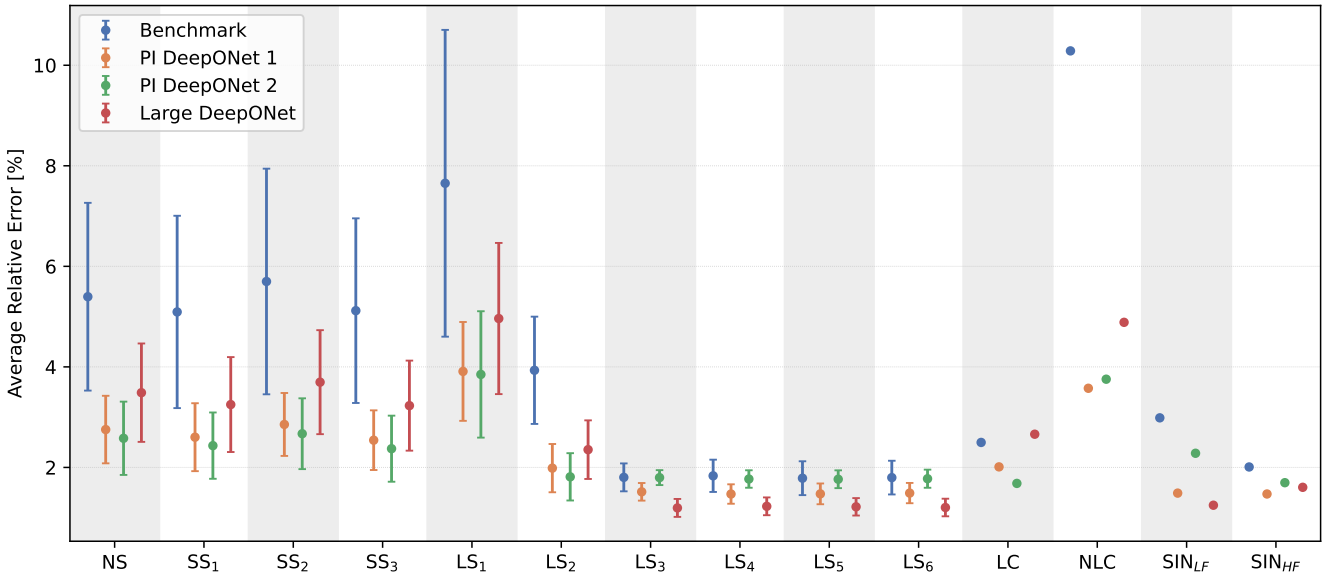

FIG. 8. ARE score of models trained on Dataset 2, the medium sized dataset of 100 source distributions $Q(x)$.

ures 7 and 8, we see a change in which models perform well. Unsurpisingly, the general performance is worse than for the models trained on the original dataset. The benchmark model performs significantly worse than the other models. We also see that the Large DeepONet is outperformed by the PI DeepONets in the NS, SS and LS1 cases. For the other large shift cases, corresponding to larger length scales, the Large DeepONet seems to perform similarly or slightly better. Comparing the predictions made on the samples shown in 12, we see that the physics-informed model is able to follow the high-frequent oscillations of the NS sample significantly better. The Large DeepONet seems to have settled on a set of parameters that gives a similar prediction different distributions – one that gives a less varying output in the middle of the slab and decreases towards zero at the boundaries. This gives an ARE that is only slightly worse for the NS, SS and LS1 cases. However, since the prediction does not reproduce the true shape of the scalar flux, its R^2 score is worse than that of the PI DeepONets. For the LS3 sample, which has a less varying scalar flux, the Large DeepONet is able to reproduce the true scalar

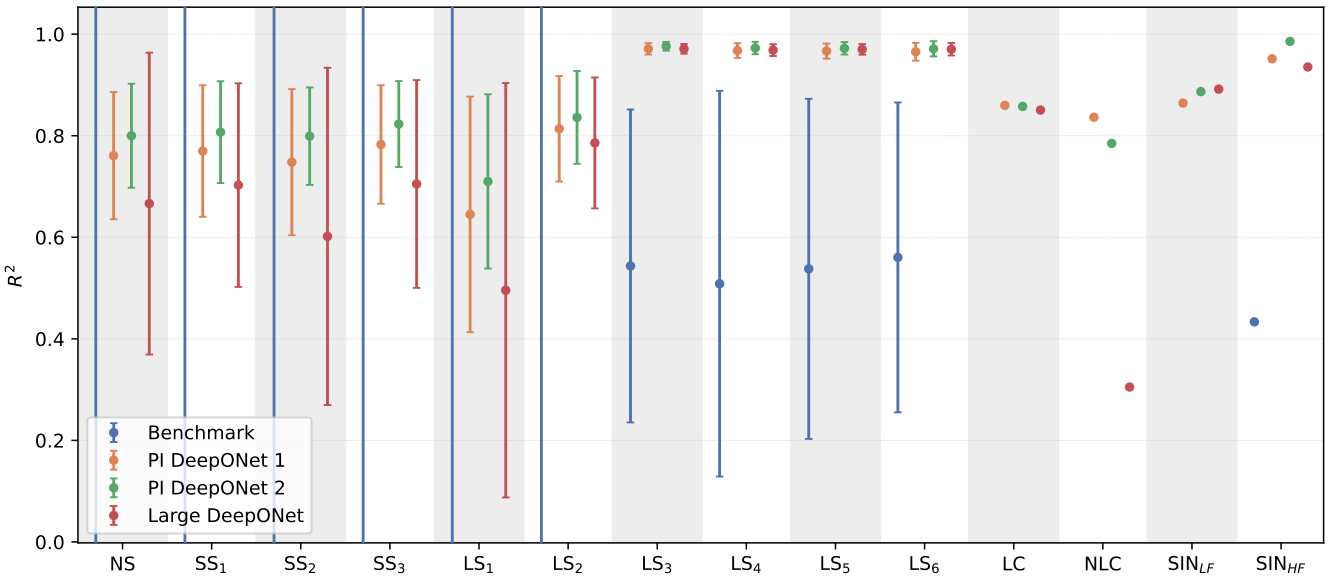

FIG. 9. R^2 score of models trained on Dataset 3, the small dataset of 10 source distributions $Q(x)$.

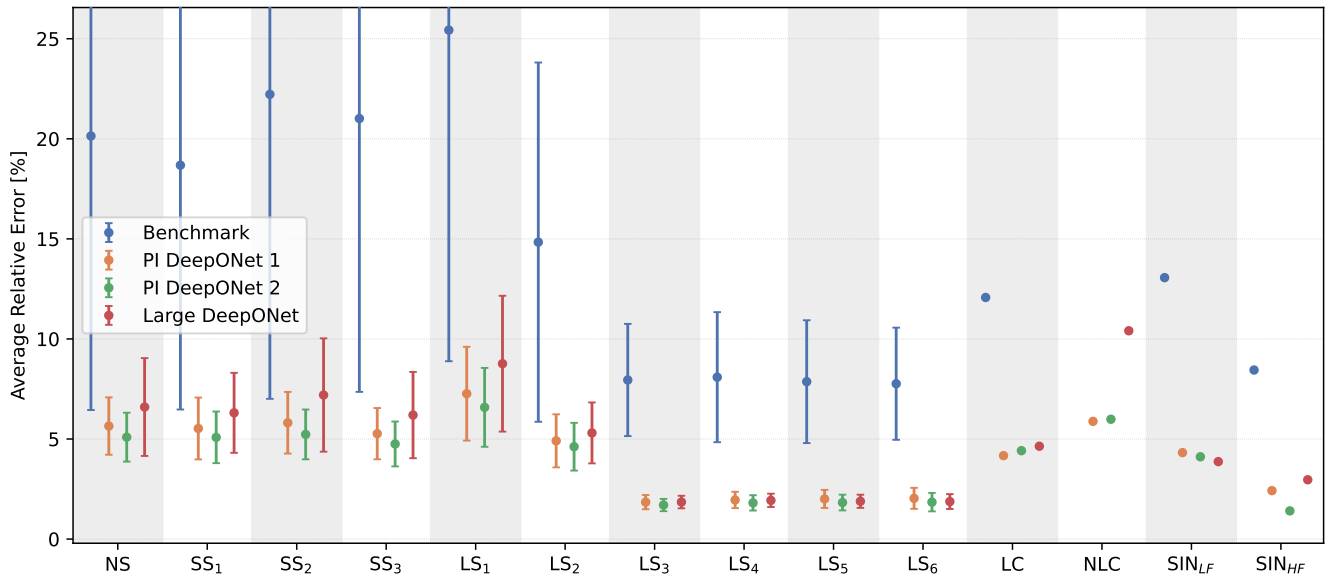

FIG. 10. ARE score of models trained on Dataset 3, the small dataset of 10 source distributions $Q(x)$.

flux better since the large length scale gives a flux closer to its basic prediction.

Finally, for the models trained on Dataset 3, we see in Figures 9 amd 10 that the performance has dropped further. The tendency we saw in the models trained on Dataset 2 become even more clear, as the models predict an even more constant scalar flux across different source distributions. This can be seen in Figure 13. We see that the physics-informed models also show more of this tendency, although their higher R^2 scores indicate that they are still able to recover more of the true physics than the Large DeepONet.

VII. CONCLUSION

In this paper, we have investigated the effect of adding physics-informed training for the performance of a DeepONet on the Neutron Transport problem of a 1D slab geometry. We were especially interested in studying to which extent physics-informed training can compensate for a lack of training data. We compared the perfor-

mance of four models. The DeepONet of Sahadath et al. [9] served as a benchmark. We developed two variants of physics-informed DeepONets; one trained on the same scalar flux data as the benchmark, the other on the corresponding angular flux data. Since the two models had more parameters than the benchmark, we also trained a DeepONet of the same size to rule out performance improvement caused by the models' size. To assess the question of how well training data can be replaced by physics-informed training, we generated three training datasets of different sizes. The largest dataset matched the size of the original dataset used by Sahadath et al. of 500 source distributions $Q(x)$ and their corresponding scalar flux $\varphi_0(x)$ (angular flux for PI DeepONet 2). The two other datasets contained 100 and 10 input-output pairs, respectively.

For the original dataset, we found that the Large DeepONet performed the best. As we decreased the size of the training datasets, we saw the physics-informed models performing better; this was most clear for the medium sized dataset. For the medium and small dataset, the Large DeepONet settled to an almost constant prediction across all source distributions. There was some tendency for the physics-informed models to do the same when trained on the small dataset, although they still performed better than the Large DeepONet.

Our results indicate that physics-informed training does in fact replace labelled training to some extent, as we would have hoped. For a large enough training dataset, however, we still see that the ordinary DeepONet performs better. It should be noted that the work in this paper is so far limited to the first problem presented by Sahadath et al., namely that of isotropic scattering. The authors also studied anisotropic scattering and pure scattering (zero absorption), in addition to training a model on a range of cross-section values to assess its ability to generalize across varying physical conditions. We plan on extending this work by considering these remaining problems. Further ahead, the methodology will be extended to consider the eigenvalue problem of a 2D or 3D reactor. Another possible direction would be to extend to time-dependent problems, opening up the possibility to consider transient scenarios.

[1] International Atomic Energy Agency, *Small Modular Reactors: Advances in SMR Developments 2024*, Tech. Rep. (International Atomic Energy Agency, Vienna, 2024).

[2] G. Black, D. Shropshire, K. Araújo, and A. van Heek, Nuclear Technology **209**, S1 (2023).

[3] J. Emblemsvåg, C. Ordonez, C. G. Tamm, J. S. Ortigosa, Y. M. Colilles, A. Pérez, H. Thoresen, E. Ovrum, M. J. F. Jordahl, L. Laanke, *et al.*, (2024).

[4] K. Wilsdon, J. Hansel, M. R. Kunz, and J. Browning, Progress in Nuclear Energy **163**, 104813 (2023).

[5] H. Mengyan, Z. Xueyan, P. Cuiting, Z. Yixuan, and Y. Jun, Annals of Nuclear Energy **202**, 110491 (2024).

[6] B. Kochunas and X. Huan, Energies **14**, 4235 (2021).

[7] P. Shen, X. Guo, K. Li, S. Huang, L. Zheng, Q. Pan, and K. Wang, Annals of Nuclear Energy **167**, 108861 (2022).

[8] F. Zhou, Y. Yang, X. Wu, P. Fang, Q. Liang, H. Lai, Y. Guo, Y. Zhu, and L. Yang, Annals of Nuclear Energy **208**, 110759 (2024).

[9] M. H. Sahadath, Q. Cheng, S. Pan, and W. Ji, Nuclear Science and Engineering , 1 (2026).

[10] S. Wang, H. Wang, and P. Perdikaris, Science advances **7**, eabi8605 (2021).

[11] S. Mousavi, "Operator learning via physics-informed DeepONet: Let's implement it from scratch," `https://medium.com/data-science/operator-learning-via-physics-informed-deeponet-lets-imple` (2023), accessed: 2026-02-16.

[12] M. Raissi, P. Perdikaris, and G. E. Karniadakis, Journal of Computational physics **378**, 686 (2019).

[13] K. Prantikos, L. H. Tsoukalas, and A. Heifetz, Energies **15**, 7697 (2022).

[14] M. H. Elhareef and Z. Wu, Nuclear Science and Engineering **197**, 601 (2023).

[15] Y. Xie, Y. Wang, and Y. Ma, Annals of Nuclear Energy **195**, 110181 (2024).

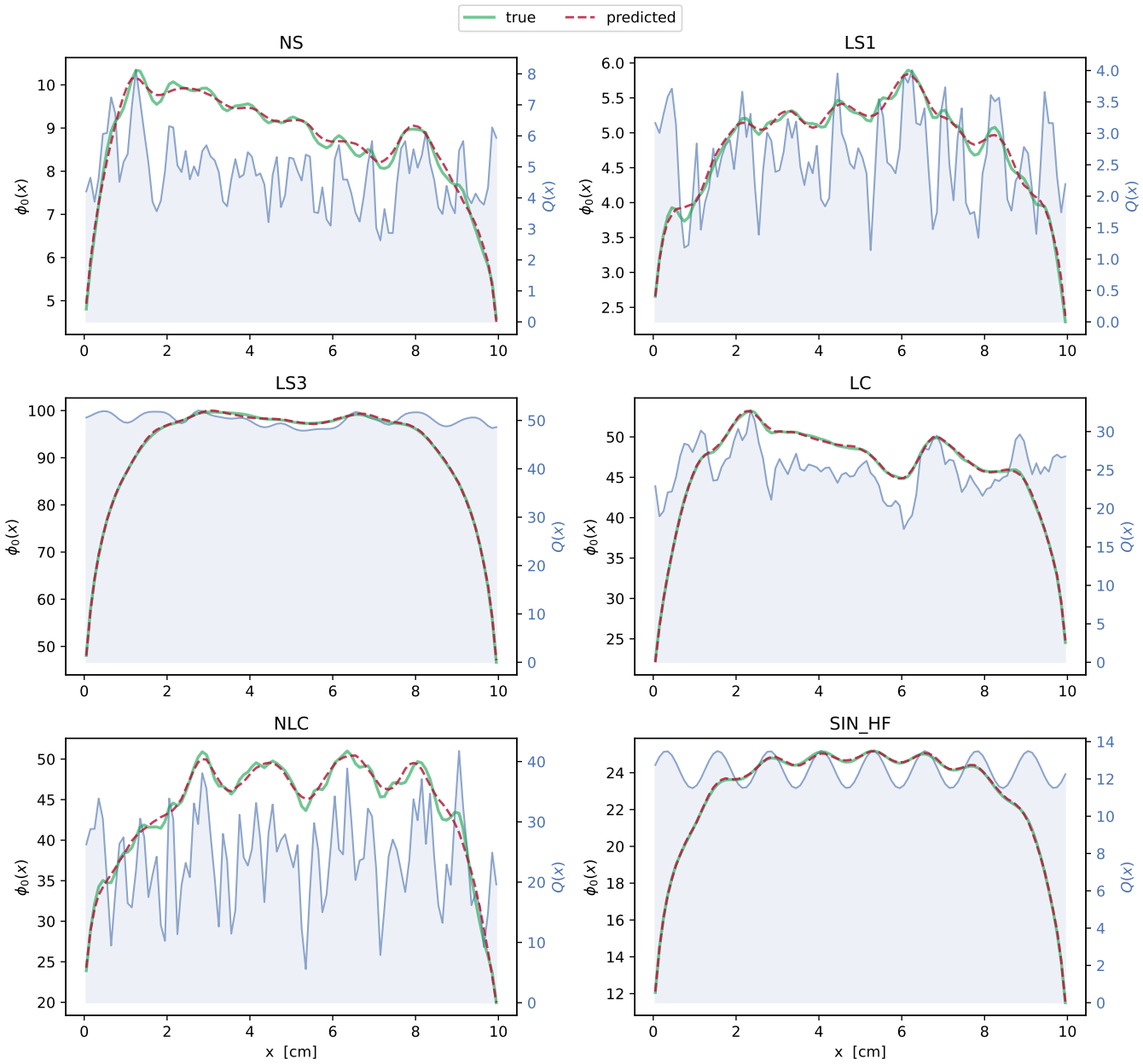

FIG. 11. Samples of predictions made by PI DeepONet 1 trained on Dataset 1 along with source distribution $Q(x)$ and true scalar flux $\varphi_0(x)$.

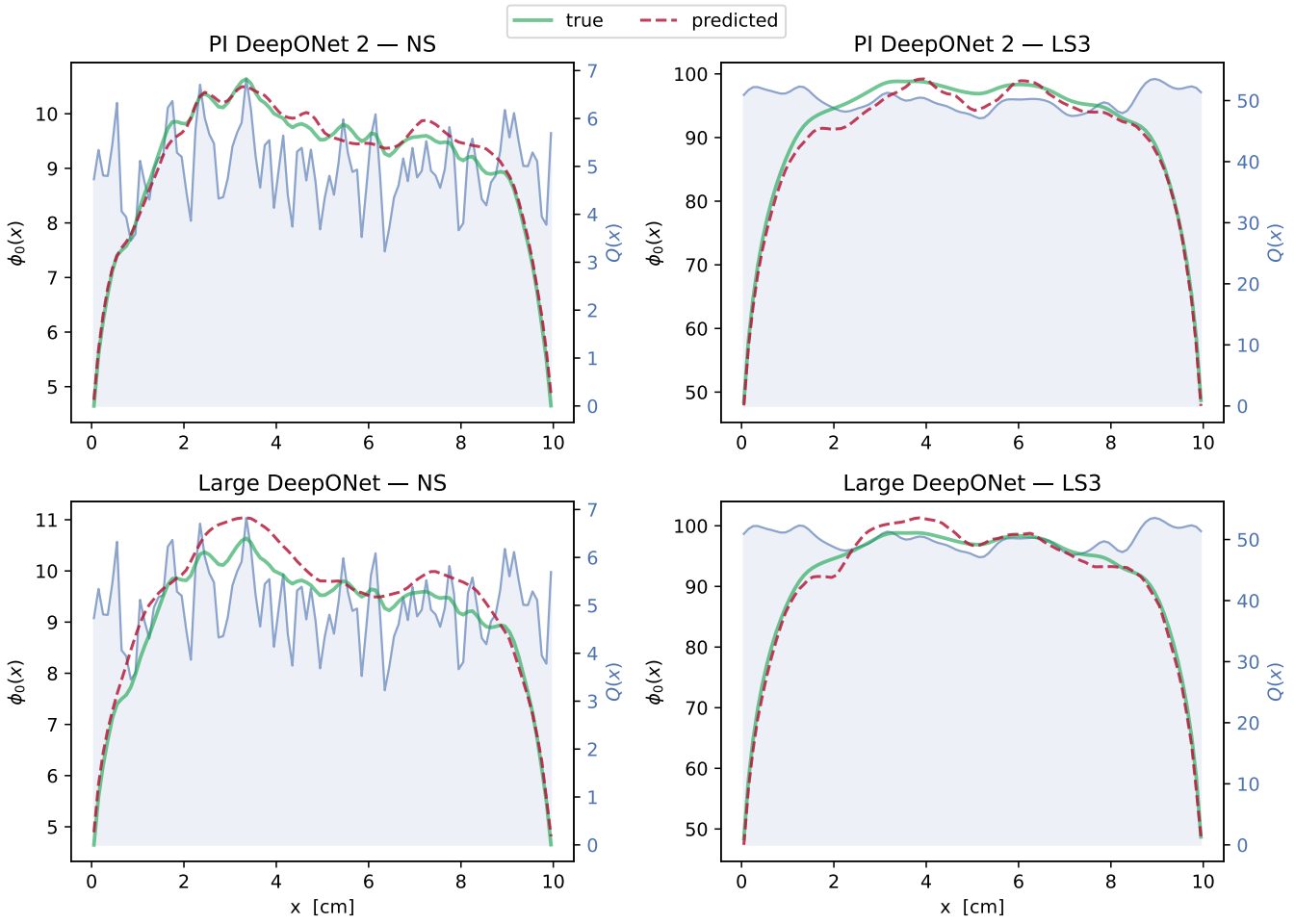

FIG. 12. Comparison of sample predictions made by PI DeepONet 2 and Large DeepONet trained on Dataset 2.

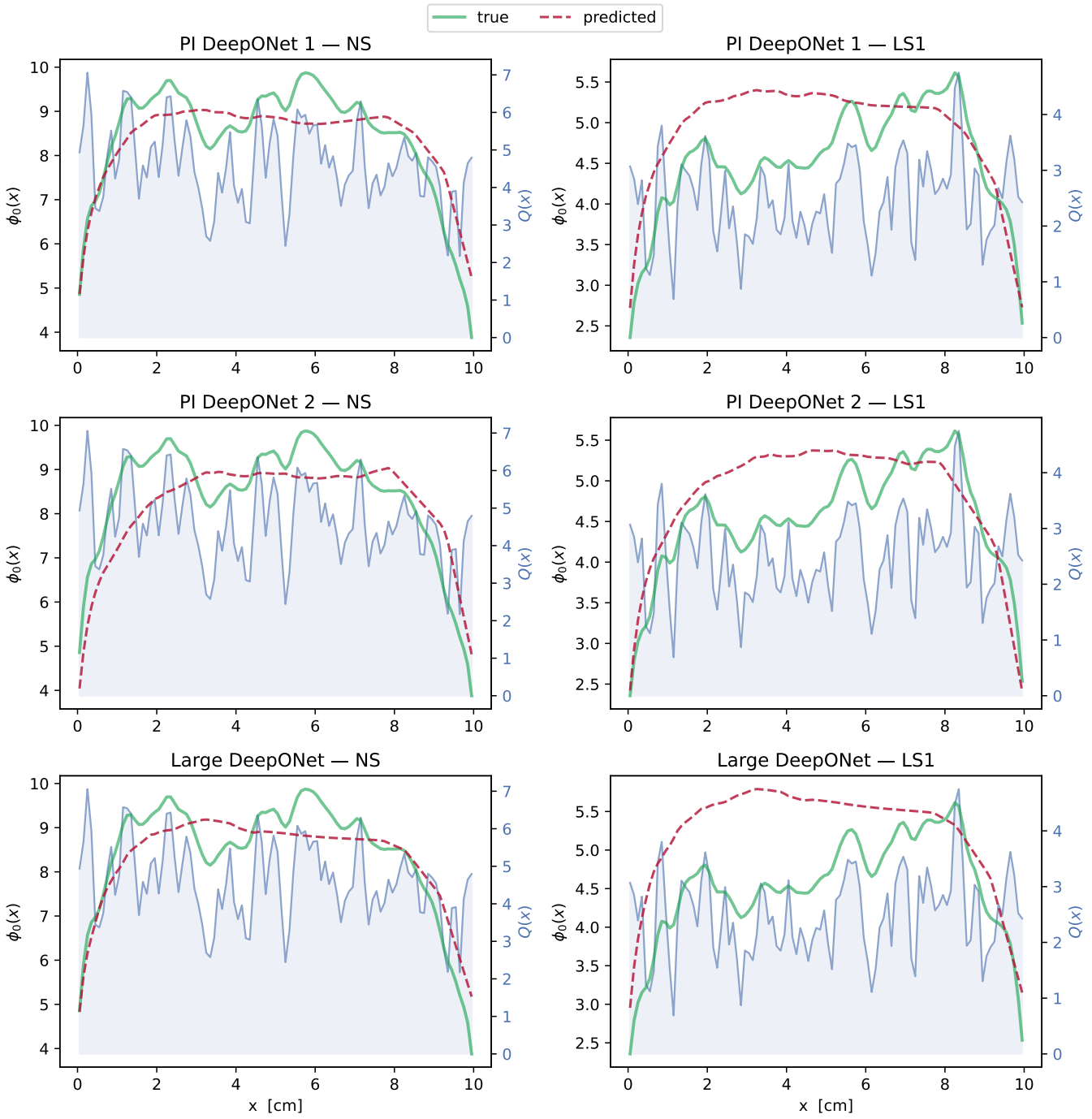

FIG. 13. Comparison of NS sample predictions made by all models on trained on Dataset 3.